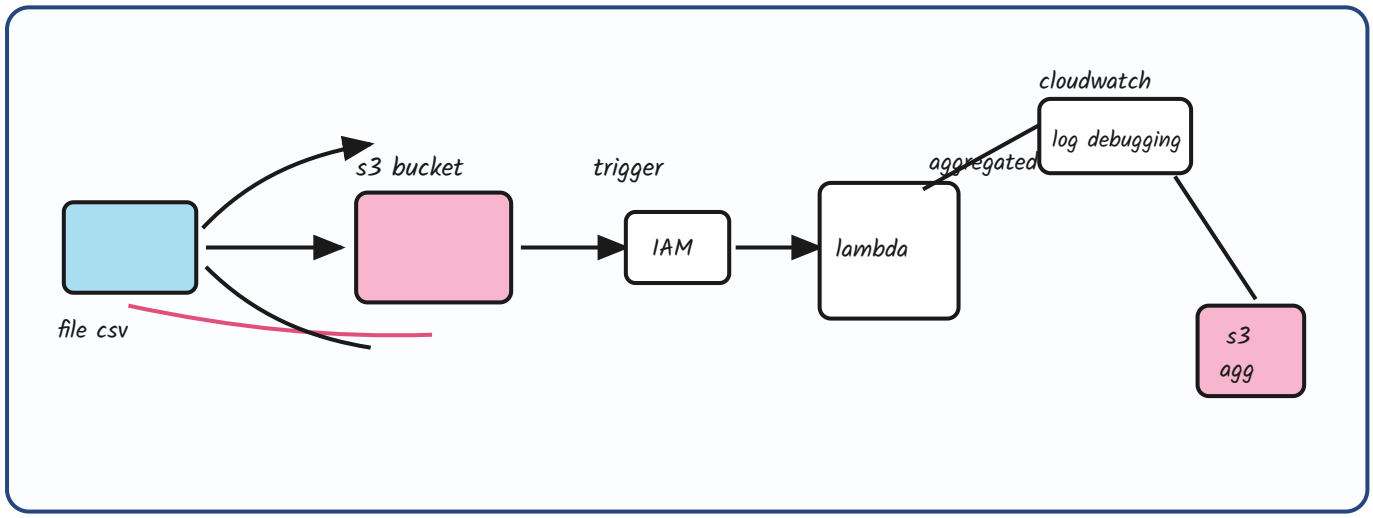
AWS Mini Project -1

event-driven employee salary aggregation on AWS

business requirements :

1. we have one of the s3 bucket where data will be coming on hourly basis
2. we should have pipeline which should be triggered whenever any file is coming to bucket
3. pipeline should be able to read the employee data and aggregate the salary data based on country .
4. aggregate data should be dump to one of the bucket.

architecture



what each piece does

- ***Source S3 bucket*** — the ***landing zone***. Employee CSV files arrive here ***every hour***. Nothing polls it; the bucket itself announces new objects.
- ***S3 event notification (trigger)*** — configured on ***s3:ObjectCreated:****. The moment a file lands, S3 pushes an event and the pipeline starts. This is what makes the design ***event-driven instead of scheduled***.
- ***IAM role*** — the ***permission layer*** between the services. The Lambda execution role needs ***read on the source bucket***, ***write on the target bucket***, and ***logs:PutLogEvents*** for CloudWatch. Least privilege: scope it to those two bucket ARNs, not `s3:*`.
- ***Lambda*** — the ***compute***. Reads the CSV out of the event payload's bucket/key, ***groups salary by country***, and writes the result out. No servers to manage; billed per invocation and duration.
- ***CloudWatch*** — ***logs and debugging***. Every print / exception from the function lands in a log group, plus metrics for ***invocations, duration, errors and throttles***.
- ***Target S3 bucket (s3 agg)*** — where the ***aggregated country-wise salary output*** is dumped. Keep it ***separate from the source bucket*** — writing back into the same bucket can retrigger the function and cause an ***infinite loop***.

Watch out: pandas is **not in the default Lambda runtime** — ship it as a **Lambda layer** or a container image. And **large files will hit the 15-minute timeout / 10 GB memory ceiling**; at that point move the aggregation to **Glue or EMR** and keep Lambda only as the trigger.

💡 Remember!

The whole flow is **S3 → event trigger → IAM role → Lambda → S3 (agg)**, with **CloudWatch watching from the side**. Three things decide whether it survives production: **separate source and target buckets** (or you loop forever), **a least-privilege IAM role** scoped to those two ARNs, and **knowing when the file is too big for Lambda** and belongs in Glue or EMR instead.